

PySpark and Spark SQL for ETL

Lazy evaluation, the core transformations, writing tables, views, and how to read code under exam conditions.

PySpark and Spark SQL are **the same engine**. Both compile to the same plan, and neither is faster. The exam expects you to read both.

Two front ends, one engine

PYSPARK DATAFRAME API

Method chaining

- `df.filter(...).groupBy(...).agg(...)`
- Composable in Python; build chains programmatically
- Errors surface as Python exceptions
- Suits complex conditional logic

SPARK SQL

Declarative statements

- `SELECT ... FROM ... WHERE ... GROUP BY`
- Familiar to analysts and BI teams
- Runs anywhere SQL runs, including warehouses
- Suits set-based transformations

THEY INTEROPERATE

You can move between them freely. `createOrReplaceTempView` exposes a `DataFrame` to SQL; `spark.sql` returns a `DataFrame`.

Transformations are **lazy**. Nothing runs until an action asks for a result. Spark spends that time optimising the whole chain at once.

Transformations and actions

TRANSFORMATIONS (LAZY)	ACTIONS (TRIGGER EXECUTION)
<code>select, filter, where</code>	<code>show, display</code>
<code>withColumn, withColumnRenamed</code>	<code>collect, take, first</code>
<code>join, union</code>	<code>count</code>
<code>groupBy with agg</code>	<code>write, saveAsTable</code>
<code>orderBy, distinct, dropDuplicates</code>	<code>toPandas</code>

ON THE EXAM

If a cell returns instantly and does nothing visible, you wrote a transformation. If it takes two minutes, you called an action.

Two consequences of laziness

- ◆ **Errors appear late.** A typo in a column name may not raise until the cell that calls an action.
- ◆ **Re-using a DataFrame re-computes it**, unless you cache. Calling three actions runs the chain three times.
- ◆ **Filters get pushed down**, so filtering after a join is often as fast as filtering before it.
- ◆ **Use explain to see the plan** Spark actually intends to run, rather than the one you wrote.

ON THE EXAM

An error surfacing in a cell you did not change usually means the mistake is in an earlier, lazy cell.

Narrow and wide transformations

- ◆ **Narrow:** each output partition depends on one input partition. select, filter, withColumn.
- ◆ **Wide:** output partitions depend on many input partitions. join, groupBy, distinct, orderBy.
- ◆ **Wide transformations shuffle**, moving data across the network between executors.
- ◆ **Shuffles are where jobs get slow.** Domain 6 is largely about diagnosing them.

ON THE EXAM

A stage boundary in the Spark UI is a shuffle. Counting the wide transformations in a chain predicts the number of stages.

The core transformations

OPERATION	WHAT IT DOES
<code>select("a", "b")</code>	Choose columns. Also renames, with alias.
<code>filter(...)</code> / <code>where(...)</code>	Keep rows matching a condition. Identical methods.
<code>withColumn("c", expr)</code>	Add a column, or replace one of the same name.
<code>withColumnRenamed("a", "b")</code>	Rename a column, leaving the data alone.
<code>drop("a")</code>	Remove a column.
<code>distinct()</code> / <code>dropDuplicates([...])</code>	Remove duplicate rows, optionally on given columns.

ON THE EXAM

`filter` and `where` are the same method with two names. Both exist so that SQL people and Python people each find one they expect.

Referring to columns

- ◆ `col("name")` refers to a column. Needed when you build expressions rather than name columns.
- ◆ `lit(5)` wraps a literal value, so Spark treats it as a constant rather than a column name.
- ◆ `expr("a + b * 2")` takes a SQL expression as a string, inside the DataFrame API.
- ◆ **A bare string is a column name.** Passing 5 where a column is expected is a common error.

ON THE EXAM

`lit` exists because Spark cannot tell a constant from a column name otherwise. It is the answer to 'why does this say column not found'.

withColumn or select?

WITHCOLUMN

Add or replace one column

- Keeps every existing column
- Replaces in place if the name already exists
- Readable for one or two additions
- Chaining many of them builds a large plan

SELECT

State the full output

- Only the columns you name survive
- Several derived columns in one step
- Clearer when the output shape changes a lot
- Easy to drop a column by forgetting it

WORTH KNOWING

withColumn with an existing name silently replaces that column. That is either exactly what you wanted or a quiet bug.

Write modes

MODE	BEHAVIOUR IF THE TARGET ALREADY EXISTS
<code>append</code>	Add these rows to the existing data
<code>overwrite</code>	Replace the existing data entirely
<code>error or errorifexists</code>	Throw an exception. This is the default.
<code>ignore</code>	Do nothing at all, silently

ON THE EXAM

The default is to fail. That is deliberate: a write that silently destroys or duplicates data would be a worse default.

saveAsTable and insertInto are not the same

- ◆ **saveAsTable matches columns by name.** The order in your DataFrame does not have to match the table.
- ◆ **insertInto matches columns by position.** Names are ignored entirely.
- ◆ **So insertInto with reordered columns writes data into the wrong columns,** silently, if the types happen to be compatible.
- ◆ **With overwrite mode, saveAsTable does not require the schema to match** the existing table.

ON THE EXAM

This is a genuine production hazard. Two string columns swapped by insertInto produce no error at all.

CTAS or create then insert?

CREATE TABLE AS SELECT

One statement

- Schema is inferred from the query result
- Table and data created together
- Cannot specify column comments or constraints inline
- Common for derived silver and gold tables

CREATE TABLE, THEN INSERT

Two statements

- Schema declared explicitly, with types you choose
- Constraints, comments and properties can be set
- Empty table exists before any data lands
- Better when the contract matters

THE TIEBREAKER

CTAS is quicker and takes the query's types. Declaring the schema is how you stop those types drifting when the query changes.

Three things called views

KIND	SCOPE	LIVES UNTIL
Temporary view	The SparkSession that created it	The session ends
Global temporary view	All sessions in the cluster, via global_temp	The cluster terminates
View (persisted)	Registered in Unity Catalog	It is dropped

ON THE EXAM

Only the third survives a cluster restart, and only the third can be granted to another user.

Views hold a query, not data

- ◆ **A view stores a query definition.** Every time you read it, the query runs again.
- ◆ **So a view is always current,** because it reads the underlying table at query time.
- ◆ **And a view is not faster.** An expensive query is expensive every time it is read.
- ◆ **A materialised view stores results,** and is the answer when the same expensive query runs repeatedly.

ON THE EXAM

View, table and materialised view differ in what is stored. Module 13 works through the choice properly.

Common table expressions

- ◆ **WITH name AS (SELECT ...)** names a subquery so the main statement reads clearly.
- ◆ **Several CTEs can be chained**, each referring to the ones before it.
- ◆ **A CTE is not stored and not reused.** Referencing it twice may compute it twice.
- ◆ **They exist for readability**, turning a nested pile of subqueries into named steps.

ON THE EXAM

A CTE is scoped to the single statement it appears in. It is not a temporary view and does not outlive the query.

DDL and DML

- ◆ **DDL defines structure:** CREATE, ALTER, DROP, TRUNCATE.
- ◆ **DML changes data:** INSERT, UPDATE, DELETE, MERGE.
- ◆ **CREATE OR REPLACE TABLE is DDL** and replaces the whole object, including its schema.
- ◆ **INSERT OVERWRITE is DML** and replaces the data while keeping the existing schema.

WORTH DISTINGUISHING

That last pair is the classic confusion. One can change the schema; the other cannot.

Reading code under exam conditions

- ◆ **Read the last line first.** What is this chain producing, and where is it going?
- ◆ **Check the order of filter and aggregate.** Filtering before or after a groupBy gives different answers.
- ◆ **Watch for a column being replaced** by a withColumn using an existing name.
- ◆ **Look for a missing action.** A chain that is never written or displayed does nothing at all.

ON THE EXAM

Most code questions differ from their distractors in one line. Find that line rather than reading all four options fully.

Tiebreakers from this module

WHEN A QUESTION SAYS

The cell returned instantly and did nothing

The error names a cell that was not changed

Columns ended up in the wrong order

Another notebook cannot see the view

The view must survive a cluster restart

The same expensive query runs all day

The write failed because the table exists

REACH FOR

A transformation. No action was called.

A lazy transformation earlier in the chain

insertInto matches by position; saveAsTable by name

A temporary view is session-scoped

A persisted view in Unity Catalog

A materialised view, not a view

Default mode is error. Choose append or overwrite.

ON THE EXAM

Seven lines. The third is the one that causes silent data corruption rather than an error.

An engineer writes a notebook cell containing a chain: read the bronze table, filter to the last 30 days, add two derived columns, and drop three columns. The cell completes in under a second.

She concludes the transformation is extremely fast and moves on. The next morning the silver table is empty.

What went wrong?

- ☐ A The filter removed all rows, because the date comparison used the wrong type
- ☐ B The chain is all transformations, so nothing executed and nothing was written
- ☐ C The bronze table was empty when the cell ran
- ☐ D Spark cached the result but did not persist it to the table

◆ PAUSE AND DECIDE

What went wrong?

A The filter removed all rows

A filter that matches nothing still takes time to run. Sub-second completion means the filter never executed at all.

C Bronze was empty at the time

Possible in principle, but it does not explain the sub-second completion, and the chain never wrote anywhere regardless.

B All transformations, so nothing ran or was written

Correct. read, filter, withColumn and drop are all lazy. With no action, Spark built a plan and did nothing else.

D Spark cached but did not persist

cache is itself lazy, and nothing here requested caching. Inventing a mechanism that was never invoked.

THE SIGNAL

'Completes in under a second' is the tell. Real work on a real table takes real time, so instant completion means no work happened.

A silver table has columns in the order: `customer_id`, `given_name`, `family_name`, `city`. A daily job builds a DataFrame with columns in the order `customer_id`, `family_name`, `given_name`, `city` and writes it with `insertInto`.

The job succeeds every night with no errors. Weeks later, support reports that customer names are being displayed back to front.

What is the cause?

- ☐ A `insertInto` matches columns by position, so the two name columns were swapped
- ☐ B The source system began sending names in the wrong order
- ☐ C The table schema was altered to reorder the columns
- ☐ D `insertInto` requires an explicit column list, and omitting it randomises the order

◆ PAUSE AND DECIDE

What is the cause?

- A

 insertInto matches by position
Correct. Unlike saveAsTable, insertInto ignores column names entirely. Both columns are strings, so nothing errors and the values land swapped.
- B

 The source changed the order
Blames the source, but the scenario tells you the DataFrame's column order directly. The mismatch is stated, not hypothetical.
- C

 The table was altered
Would produce the same symptom, but the scenario already supplies a sufficient cause. Do not invent a second one.
- D

 Omitting a column list randomises order
Fabricated. Positional matching is deterministic, which is exactly why it fails consistently rather than randomly.

THE SIGNAL

No error, for weeks, with a real mismatch stated in the setup. Silent wrongness of that kind points at positional matching.

An analyst builds a complex filtered and joined dataset in a notebook and registers it with `createOrReplaceTempView`. She tells a colleague to query it from his own notebook.

He reports that the view does not exist. She can see it perfectly well from hers. The next morning she cannot see it either.

What is happening, and what should they use?

- A A permissions problem; he needs `SELECT` granted on the view
- B A temporary view is scoped to one `SparkSession` and dies with it; they need a persisted view or a table
- C He needs to query it as `global_temp.view_name`
- D The view must be refreshed each morning with `REFRESH TABLE`

◆ PAUSE AND DECIDE

What is happening, and what should they use?

A Grant him SELECT

There is nothing to grant. A temporary view is not a Unity Catalog securable, so no permission would make it visible to him.

C Query it as `global_temp.view_name`

That is the syntax for a global temporary view, which this is not. It would also still die with the cluster.

B Session-scoped; use a persisted view or table

Correct. It explains both symptoms at once: invisible to his session, and gone from hers after a restart.

D REFRESH TABLE each morning

Refresh updates cached metadata for something that exists. It cannot recreate an object that no longer exists.

THE SIGNAL

Two symptoms, one cause. Any answer explaining only the colleague's problem leaves the overnight disappearance unexplained.

A gold table is defined with CREATE TABLE AS SELECT over a silver table. Six months later a colleague changes the silver query so that a total_value column, previously computed as an integer, is now computed with a decimal division.

The gold table has been rebuilt nightly with CREATE OR REPLACE TABLE. A downstream BI report that summed total_value now shows different numbers.

What is the underlying design weakness?

- ☐ A CREATE OR REPLACE TABLE should have been INSERT OVERWRITE
- ☐ B CTAS takes its column types from the query, so a change to the query silently changed the table's schema
- ☐ C The BI tool cached the old schema and needs refreshing
- ☐ D Gold tables should never be rebuilt from silver

◆ PAUSE AND DECIDE

What is the underlying design weakness?

- A

 Should have used INSERT OVERWRITE
Close, and worth understanding. INSERT OVERWRITE keeps the existing schema, so it would have failed loudly instead of changing types silently.
- B

 CTAS inherits types from the query
Correct. The schema was never declared, so it is whatever the query produces, and changing the query changed the contract.
- C

 The BI tool cached the schema
The numbers changed because the values changed. A stale cache would show old numbers, not new ones.
- D

 Gold should not be rebuilt from silver
Rebuilding gold from silver is the normal medallion pattern. Nothing about it caused this.

THE SIGNAL

A and B describe the same failure from two angles. B names the weakness; A names the mechanism that would have caught it.

Nothing runs until an action asks for a result. **Everything else in this domain is built on that.**